

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 –DAST SIMULATION

Course Code:	CSA5803	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 2
Weightage:	30% of CIA	Total Marks:	100
CO3 Statement:	Simulate Dynamic Application Security Testing (DAST) and Software Composition Analysis (SCA).		
Student Name:	Lokesh kumar v	Reg. No.:	192521170
Section / Batch:	c	Date:	13/07/26

Instructions to Students

- Ensure all diagrams and workflow illustrations are **original, clear, and professionally formatted**.
- Include **all editable source files** (such as .yaml, .py, requirements.txt, .pre-commit-config.yaml, and any diagram source files if applicable).
- Submit **both** the **GitHub repository link** and the **final PDF report** through **Google Classroom** before the specified deadline.
- Verify that the GitHub repository is accessible and contains all required project files, screenshots, and documentation.

Question Type	Focus Area	Questions	Marks
DAST SIMULATION	Application based	Q1	Q1 –100
GRAND TOTAL:		100 Marks	

Code of Conduct

I, lokesh kumar(192521170) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

lokesh
Signature of the Student:

RUBRICS FOR CALCULATION

Simulate DAST and SCA Using a Real-World Scenario

Scenario: Online Banking Application

A bank has developed an online banking web application that allows customers to log in, check balances, and perform transactions. Before production deployment, the security team performs:

- **DAST (Dynamic Application Security Testing)** – tests the running application from the outside.
- **SCA (Software Composition Analysis)** – checks third-party libraries and dependencies for known vulnerabilities.

A. DAST – Dynamic Application Security Testing

1. Aim

To simulate DAST on a running online banking application and identify vulnerabilities that can be detected during runtime.

2. Test Environment

Application: Online Banking Web Application

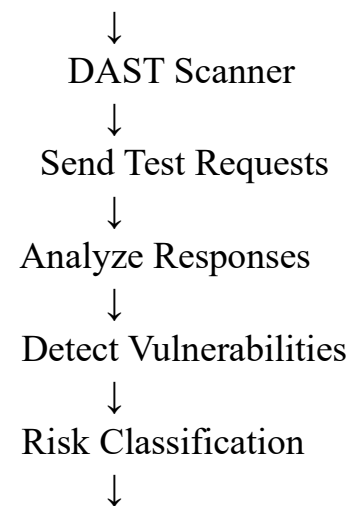
URL: <http://banking-test.local>

Environment: Test/Staging Server

Testing Type: Black-box

3. DAST Workflow

Running Web Application



Developer Fixes



Re-test

4. Simulated DAST Results

ID	Test	Finding	Severity
DAST-001	Login testing	Weak authentication controls	High
DAST-002	Input testing	Cross-Site Scripting (XSS)	High
DAST-003	Session testing	Improper session management	High
DAST-004	Error testing	Detailed error information exposed	Medium
DAST-005	Access testing	Missing authorization check	Critical

5. Example Finding

Vulnerability: Cross-Site Scripting (XSS)

Observation:

The application displays user-supplied input without properly encoding it.

Risk:

Malicious input could be interpreted as executable browser content.

Recommendation:

- Validate user input.
- Encode output.
- Apply appropriate Content Security Policy controls.

B. SCA – Software Composition Analysis

1. Aim

To identify vulnerabilities in third-party libraries used by the banking application.

Modern applications often depend on external packages. SCA checks these dependencies against known vulnerability databases.

2. Synthetic Dependency File

Example requirements.txt:

Flask==2.0.0

requests==2.19.0

Jinja2==2.10

urllib3==1.25.0

PyYAML==5.3

3. Simulated SCA Scan

Package	Version	Finding	Severity	Recommendation
Flask	2.0.0	Outdated dependency	Medium	Upgrade
requests	2.19.0	Known security issues	High	Upgrade
Jinja2	2.10	Known security issues	High	Upgrade
urllib3	1.25.0	Known security issues	High	Upgrade
PyYAML	5.3	Known security issues	Critical	Upgrade

These package findings are **synthetic examples for the assignment**; they are not claims about the exact current vulnerability status of those versions.

4. SCA Remediation

Update vulnerable dependencies to supported secure versions after checking compatibility and current security advisories.

Example:

Before:

Flask==2.0.0

requests==2.19.0

Jinja2==2.10

urllib3==1.25.0

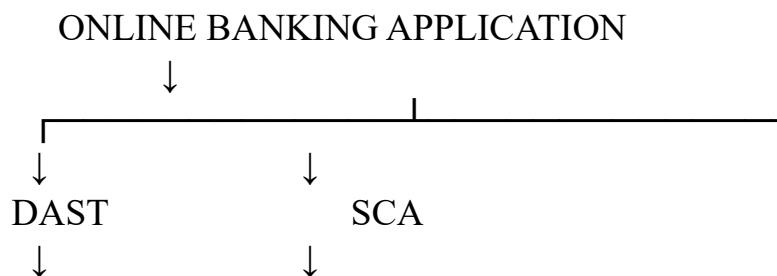
PyYAML==5.3

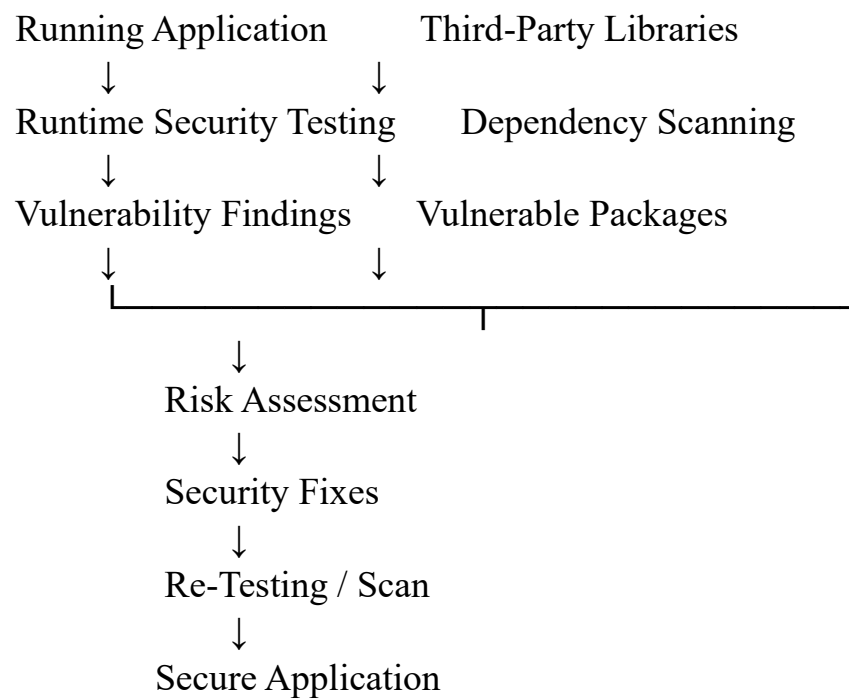
↓ SCA Analysis

After:

Use current supported secure versions
and regularly update dependencies.

5. Combined DAST + SCA Workflow





6. DAST vs SCA

Feature	DAST	SCA
Full Form	Dynamic Application Security Testing	Software Composition Analysis
Tests	Running application	Third-party components
Approach	Black-box	Dependency analysis
Finds	XSS, authentication, session issues	Vulnerable libraries/packages

Feature	DAST	SCA
Requires running application	Yes	No
Main objective	Runtime security	Supply-chain security

7. Expected Results

DAST

- 1 Critical
- 3 High
- 1 Medium

SCA

- 1 Critical
- 3 High
- 1 Medium

The combined testing identifies vulnerabilities in both the **application's runtime behavior** and its **third-party software dependencies**.

8. Conclusion

DAST and SCA provide complementary security testing. **DAST** identifies vulnerabilities in a running online banking application, while **SCA** identifies security risks in third-party libraries and dependencies. Using both techniques helps developers detect vulnerabilities earlier, reduce security risks, and improve the overall security of the application before production deployment.